TECHNICAL GUIDE

PharmaCare + PharmaIQ Technical Guide

Architecture, data science, ML / NLP,
RAG and LLM integration

Project: PharmaCare + PharmaIQ (Flask · SQLite · pandas · hybrid RAG · optional LLM)
Document date: 15 October 2026 · Based on the code and demo data in PharmaCare-PharmaIQ.zip

Contents

1. Overview and design principles . . . 3

1.1 What kind of “AI” is this? (an honest summary) . . . 3

2. Technology stack . . . 4

3. System architecture . . . 5

3.1 Module responsibilities . . . 6

3.2 Project layout . . . 6

4. Data model . . . 7

5. Request lifecycle . . . 8

5.1 JSON API . . . 9

6. Understanding the question (planning and NLU) . . . 11

6.1 Entity extraction (nlp.py) . . . 11

6.2 LLM planner (optional) . . . 11

7. Tool calling . . . 12

8. Text-to-SQL and SQL security . . . 13

8.1 The five-layer SQL guard . . . 13

9. Retrieval-Augmented Generation (RAG) . . . 15

9.1 Indexing . . . 15

9.2 Text normalisation (rag/text.py) . . . 15

9.3 Hybrid retrieval: two complementary rankers . . . 16

9.4 Rank fusion, relevance gate and focus . . . 16

9.5 Answer generation . . . 17

9.6 RAG access control and evaluation . . . 17

10. Data science: forecasting, anomalies and analytics . . . 18

10.1 Demand forecasting . . . 18

10.2 Anomaly detection . . . 19

10.3 Other analytics . . . 20

11. Where each “AI” technique is used . . . 20

12. LLM integration . . . 20

12.1 Data minimisation and failure modes . . . 21

13. Safety, security and privacy . . . 22

14. Evaluation and testing . . . 22

15. Configuration reference . . . 23

16. Extending the system and known limitations . . . 24

16.1 Common extensions . . . 24

16.2 Known limitations found during this review 24

Appendix A. Glossary of techniques 25

1. Overview and design principles

PharmaCare is a Flask + SQLite pharmacy management system (inventory, sales, alerts, barcode, reports, three user roles). **PharmaIQ** is an AI layer added as a Flask blueprint (`pharmaiq/`, about 4,020 lines of Python across 23 modules, plus 5 page templates and one JavaScript client). It lets users query the pharmacy in natural language and returns answers that are **grounded** in either the live database or approved documents.

The central engineering problem is *trust*. In a pharmacy, a plausible-sounding but wrong number or a hallucinated dosing statement is worse than no answer. The system is therefore built around a set of explicit principles, each of which is traceable to specific code:

Principle	How it is implemented
Grounded answers only	Numbers come from SQL executed on the real database; text comes from retrieved document passages. If nothing relevant is found the answer is “I won’t guess” (`rag/answer.py`).
The LLM is optional and untrusted	Default mode is fully offline rules. If an LLM is enabled it can only (1) propose a plan that is validated and (2) paraphrase verified results; its numbers and citations are checked (`planner.py`, `agent.py`).
Defence in depth for SQL	Five independent layers (lexical validation, SQLite authorizer, row-level views, read-only connection, resource limits) in `sql_guard.py`. 12 of 12 attack cases are blocked in the evaluation suite.
Role-based data access	A per-role allow-list of tables / columns / tools; staff see only their own sales rows (`permissions.py`).
Explainability	Every answer carries facts, evidence tables, the exact SQL + parameters, as-of date, assumptions, caveats and sources.
Safety boundary	A pre-filter refuses personal dosing / diagnosis / credential requests before any planning or LLM call (`safety.py`).
Auditability and testability	Every request is logged (PII-redacted); 89 unit tests and a 51-case evaluation harness verify behaviour.
Additive and non-destructive	PharmaIQ only adds tables (`CREATE TABLE IF NOT EXISTS`) and never writes to the original ones.

1.1 What kind of “AI” is this? (an honest summary)

Because this section of the guide asks how data science, ML, LLM and RAG are used, it helps to be precise about what is and is not present:

Technique	Present?	Where / how
Statistical forecasting	Yes	Damped-trend exponential smoothing with weekday seasonality, fitted per request with numpy / pandas (`analytics.py`). No pre-trained model file.
Robust anomaly detection	Yes	Median / MAD z-score on daily sales plus 7 rule-based data-quality checks.
Classical ML / information retrieval	Yes	scikit-learn `TfidfVectorizer` (character n-grams) + a hand-written BM25, fused with Reciprocal Rank Fusion.
Neural embeddings	Optional	sentence-transformers (MiniLM) + FAISS when `PHARMAIQ_EMBEDDINGS=sentence-transformers`; default is TF-IDF vectors.
RAG (retrieval-augmented generation)	Yes	Chunk → index → hybrid retrieve → gate → answer. The “generation” step is extractive by default and LLM-written only if enabled.
Large language model	Optional	Anthropic or any OpenAI-compatible endpoint for planning and explanation; off by default.

Technique	Present?	Where / how
Model training / fine-tuning	No	Nothing is trained or fine-tuned; there are no model weights in the repository.
Natural-language understanding	Rules	Regex / keyword planner with fuzzy matching against vocabulary loaded from the live database.

2. Technology stack

Layer	Technology	Purpose
Language / runtime	Python 3.9+	Entire backend and AI layer
Web framework	Flask ≥ 3.0, Werkzeug ≥ 3.0	Routing, sessions, Jinja2 templates, blueprint for PharmaIQ; Werkzeug password hashing
Database	SQLite (stdlib `sqlite3`)	Single file `instance/pharmacy.db`; raw SQL, no ORM; authorizer + progress-handler hooks used for security
Data science	pandas ≥ 2.0, numpy ≥ 1.24	Daily series construction, forecasting maths, anomaly statistics
ML / IR	scikit-learn ≥ 1.3	`TfidfVectorizer(analyzer="char_wb", ngram_range=(3,5), sublinear_tf=True)`
Retrieval (custom)	Hand-written BM25, RRF, relevance gate	No vector-DB service; chunks held in SQLite and cached in memory per role
Documents	python-docx, pypdf; optional PyMuPDF	DOCX and PDF text extraction (PyMuPDF preferred, pypdf fallback), TXT / MD parsing
Spreadsheets	openpyxl ≥ 3.1	Excel export of the original app and of AI results
Config	python-dotenv	`.env` file; every setting overridable by environment variable
Optional embeddings	sentence-transformers ≥ 2.7, faiss-cpu ≥ 1.8	Semantic search (all-MiniLM-L6-v2, ~90 MB download on first use)
Optional LLM	Anthropic Messages API or OpenAI-compatible `/chat/completions`	Called through `urllib` (no vendor SDK dependency); temperature 0
Front end	Bootstrap 5.3.3, Bootstrap Icons 1.11.3, Chart.js 4.4.0, QuaggaJS 0.12.1, vanilla JS (`ai.js`)	Layout, charts, barcode scanning, chat client (fetch + CSRF header); loaded from CDNs
Testing	unittest / pytest	89 tests in 6 files + 51-case evaluation harness

> All required dependencies are pure-pip wheels; the optional ones (PyMuPDF, sentence-transformers, FAISS) are separately listed in `requirements-optional.txt` and the app degrades gracefully without them (it reports the fallback in the Knowledge Base page header).

3. System architecture

The browser talks to Flask over ordinary session-cookie HTTP. The original application and the PharmaIQ blueprint share the same Flask app, session and SQLite file. All AI logic lives in the `pharmaiq` package behind a single orchestrator (`agent.py`).

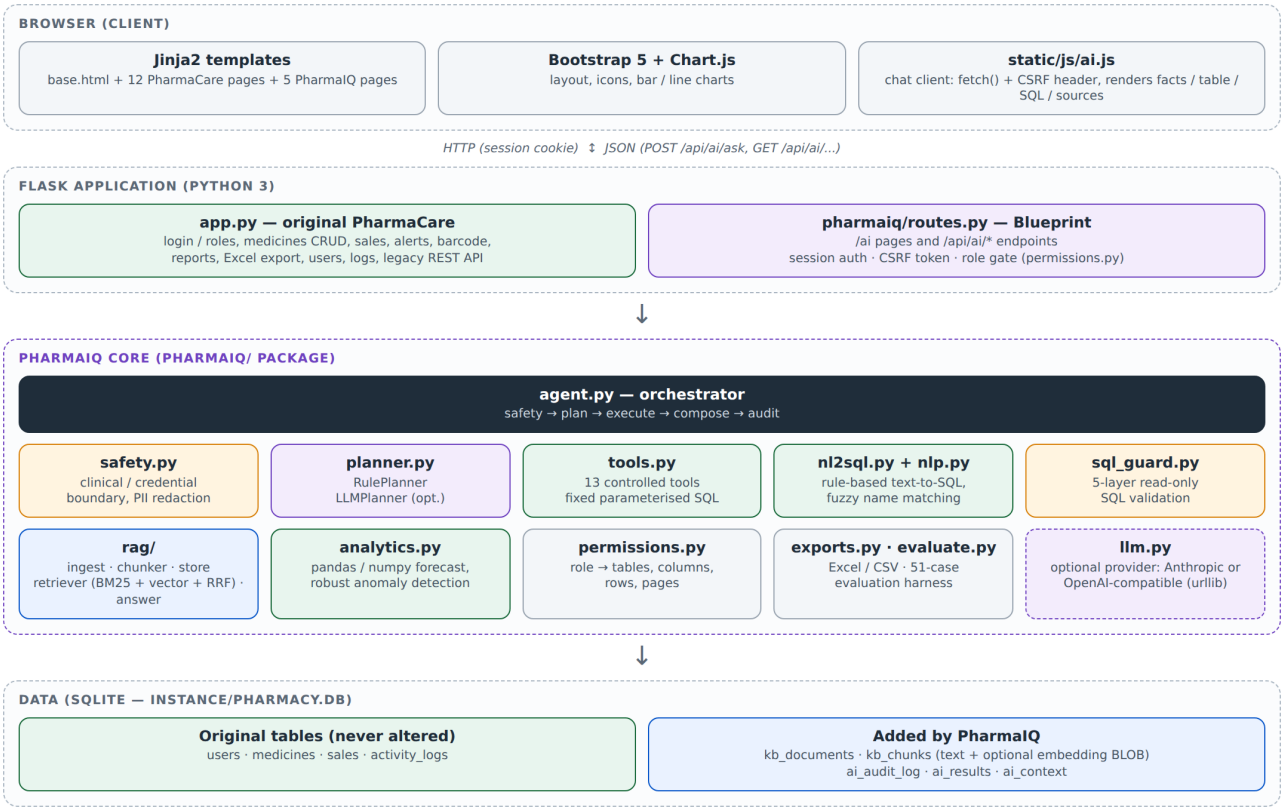


Figure 1 – Layered architecture of PharmaCare + PharmaIQ.

3.1 Module responsibilities

Module	Lines	Responsibility
agent.py	457	Orchestrates one request: safety → plan → execute → compose → persist/audit; LLM explanation, grounding check, RAG answer assembly
planner.py	333	RulePlanner (deterministic) and LLMPlanner (JSON plan, validated); follow-up merging
tools.py	677	13 tools with parameter clamping; each returns a ToolResult (tables, facts, notes, chart, follow-ups)
nlp.py / nl2sql.py	153 / 229	Entity extraction (category, company, medicine, period, numbers) and rule-based text-to-SQL
sql_guard.py	336	Lexical validator, authorizer, ReadOnlySession (row filters, timeouts, row cap)
permissions.py	92	Role → tables / columns / row filters / tools / pages; schema prompt for the LLM
safety.py	97	Clinical + credential refusal rules; PII redaction for logs
rag/ (7 files)	568	Ingest, chunk, store, BM25 + vector retrieval, fusion, relevance gate, extractive answer
analytics.py	168	Forecasting and anomaly detection (pure functions on pandas Series)
llm.py	153	Provider abstraction (Anthropic, OpenAI-compatible), robust JSON extraction
routes.py	325	Blueprint: pages, JSON API, CSRF, uploads, exports, evaluation endpoint
exports.py / evaluate.py / db.py / config.py	59 / 159 / 105 / 85	Excel / CSV (with formula-injection defence); evaluation harness; table creation + audit helpers; settings

3.2 Project layout

```
PharmaCare-PharmaIQ/
├── app.py                    original PharmaCare app (814 lines) + PharmaIQ blueprint
├── seed_data.py              30 demo medicines
├── seed_ai_demo.py           demo sales (~1,700 rows, planted anomalies) + documents
├── pharmaiq/                 the AI package (agent, planner, tools, sql_guard, rag/ ...)
├── templates/                12 original pages + 5 PharmaIQ pages (ai_*.html)
├── static/js/ai.js, static/css/ai.css   chat client and styles
├── knowledge_base/           12 sample leaflets and SOPs (Markdown, marked SAMPLE)
├── evaluation/               sql_eval.json, rag_eval.json, safety_eval.json
├── tests/                    89 tests (agent, api, evaluation, rag, sql_guard, tools)
└── requirements*.txt, .env.example, docs/
```

4. Data model

The original four tables are untouched. PharmaIQ adds five tables with `CREATE TABLE IF NOT EXISTS` at start-up, so existing databases upgrade in place.

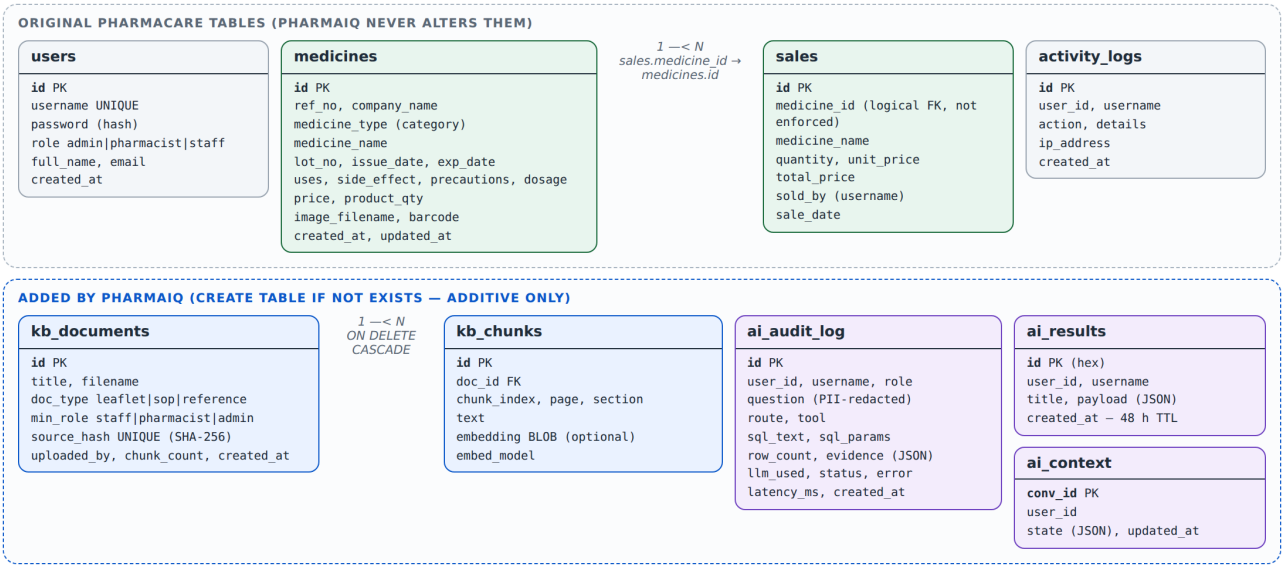


Figure 2 – Entity relationships. `sales.medicine_id` *is a logical (not enforced) reference; a data-quality check flags orphans.*

Table	Role in the AI layer
medicines, sales	The two analytical tables the SQL / tool layer reads. *medicine_type* acts as the category; *exp_date* and *product_qty* drive expiry / stock logic; *sale_date*, *quantity*, *total_price*, *sold_by* drive sales analytics and row-level security.
users, activity_logs	*users* is never readable by the AI (not in any role's allow-list). *activity_logs* is readable by admin only, without *ip_address* / *user_id*.
kb_documents / kb_chunks	Document registry (type, minimum role, SHA-256) and its chunks (text, page, section; optional embedding BLOB and model name). Deleting a document cascades to its chunks.
ai_audit_log	One row per question: user, role, redacted question, route, tool, SQL + params, row count, evidence JSON, LLM used, status, error, latency.
ai_results	Result payloads kept for Excel / CSV export; expire after 48 hours (`PHARMAIQ_RESULTS_TTL_HOURS`).
ai_context	Per-conversation state (last plan / SQL spec) enabling follow-ups; keyed by conversation id and user.

5. Request lifecycle

A question travels through six stages. Stage 3 chooses exactly one *route*; stage 4 executes it inside a `ReadOnlySession` that is bound to the caller's role and username, so no code path can run with elevated rights.

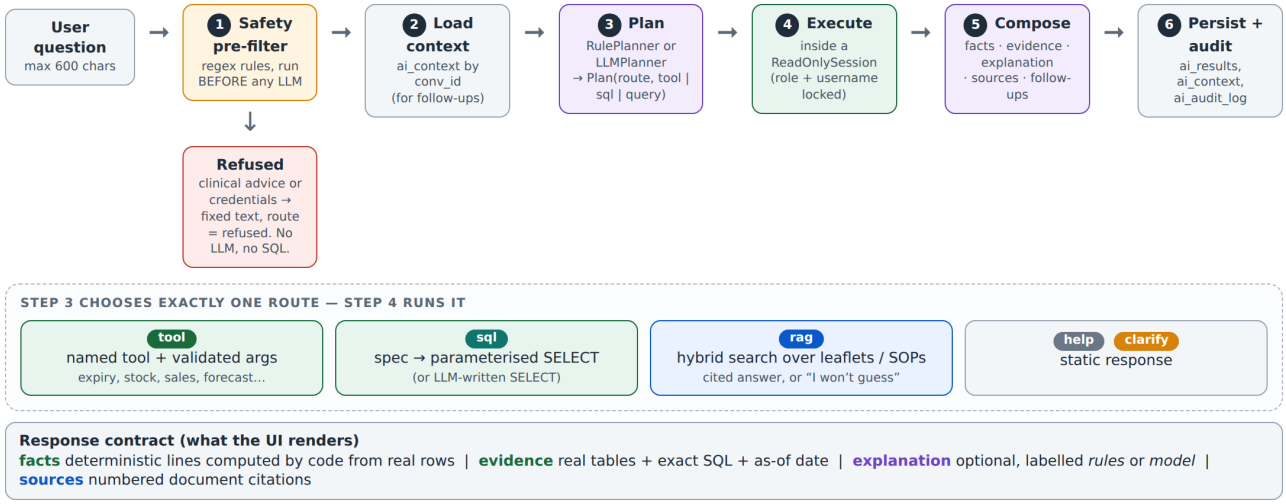


Figure 3 – End-to-end processing of a question.

Stage	What happens	Code
1 Safety pre-filter	Regex rules detect personal dosing / diagnosis / pregnancy advice, recommendations, personal numbers (kg, age) and credential requests. Match → fixed refusal text, status = refused, **no LLM / SQL is invoked**. Applies to every role (the demo admin is refused too).	safety.check_question
2 Load context	If a conversation id is supplied, the previous plan / SQL spec is loaded from *ai_context* (only for the same user).	agent.ask
3 Plan	LLMPlanner if a provider is configured (falls back on any failure), otherwise RulePlanner. Output: Plan(route, tool+args | sql | query | clarify).	planner.py
4 Execute	tool → `run_tool` (role check, argument clamping, parameterised SQL). sql → guard + execute. rag → hybrid retrieval. help / clarify → static text.	tools.py, sql_guard.py, rag/
5 Compose	Facts (deterministic), evidence tables with SQL and as-of date, optional explanation (rules or LLM, grounded), sources, 3 follow-up suggestions, chart spec.	agent._compose
6 Persist + audit	Result stored for export (48 h), context saved, audit row written (question redacted, status, latency).	db.py

5.1 JSON API

Endpoint	Method	Purpose / access
/api/ai/ask	POST	Ask a question (JSON `{question, conversation_id}`, CSRF header required) → response below
/api/ai/tools	GET	Tools available to the caller's role
/api/ai/health	GET	Provider, retrieval backend and document counts
/api/ai/export/<id>.<csv|xlsx>	GET	Download a stored result (owner only, 48 h)
/api/ai/knowledge, /api/ai/knowledge/search	GET	List documents visible to the role; run a retrieval query
/ai/knowledge/upload, /delete/<id>, /load-samples	POST	Upload (admin, pharmacist), delete and load samples (admin)
/api/ai/insights/forecast | anomalies | reorder	GET	Analytics endpoints (admin, pharmacist)
/api/ai/evaluate	POST	Run the evaluation suites (admin)
/ai, /ai/knowledge, /ai/insights, /ai/audit, /ai/evaluation	GET	The five HTML pages (role gated by permissions.PAGE_ROLES)

Sample response for *"Show low stock items"* (live output from the demo database; rows and long strings abbreviated):

```
{
 "route": "tool",
 "tool": "get_low_stock_items",
 "planner": "rules",
 "answer": "6 product(s) are at or below 10 units.",
 "facts": [
  "6 product(s) are at or below 10 units."
 ],
 "assumptions": [
  "Rule: low stock means quantity ≤ 10 units (same rule as the Alerts page)."
 ],
 "warnings": [],
 "followups": [
  "Which of these should I reorder first?",
  "Now show only capsules",
  "Show items with fewer than 5 units"
 ],
 "latency_ms": 11,
 "evidence": {
  "as_of": "2026-10-15",
  "tables": [
   {
    "title": "Low-stock items (≤ 10 units)",
    "columns": [
     "medicine_name",
     "category",
     "company"
    ],
    "rows": "[6 rows]",
    "sql": "SELECT medicine_name, medicine_type AS category, company_nam...",
    "params": {
     "t": 10,
     "limit": 200
    }
   }
  ]
 },
 "explanation": {
  "generated_by": "rules",
  "text": "Review these with the pharmacist and place orders for fast-moving items first."
 },
 "result_id": "<48h export handle>",
 "conv_id": "<conversation id>"
}
```

6. Understanding the question (planning and NLU)

The first AI decision is **what kind of question this is**. The default `RulePlanner` is a deterministic cascade; the first matching rule wins. Determinism is a feature: the same question always follows the same path, which makes behaviour testable and auditable.

#	If the (normalised) question matches…	…the RulePlanner returns
1	greeting / "what can you do"	help
2	conversation state exists AND message looks like a follow-up ("now…", "only…", "those")	refine the PREVIOUS plan / SQL spec (keep filters, change what user changed)
3	knowledge words (policy, SOP, side effects, storage, warnings…) and not about sales	rag documents first
4	anomaly / unusual / spike / data quality	tool detect_anomalies
5	forecast / predict / demand for	tool forecast_demand (horizon from "N days/weeks", default 14)
6	reorder / restock / replenish	tool get_reorder_suggestions
7	summary / overview / report / briefing	tool get_management_summary
8	inventory value / stock value / worth	tool get_inventory_value (by category / company / total)
9	sales words (sold, revenue, sales…) → sub-router	one medicine → sql; top sellers → get_top_selling; by category → get_category_sales; else get_sales_summary
10	"expiring before/after YYYY-MM-DD"	sql (tools use day-windows)
11	expiry words	tool get_expiring_medicines or get_expired_medicines
12	low stock / out of stock / running low	tool get_low_stock_items (unless a numeric comparison is present)
13	numeric filter, aggregation, grouping, ordering	sql rule-built spec → text-to-SQL
14	a medicine name was recognised	info words → rag; otherwise get_stock_level
15	category / company only, or generic "how many / in stock"	tool search_medicines
16	nothing matched	rag fallback — may answer "I won't guess"

Checked top to bottom — first match wins. Entities (category, company, medicine, period, limit, days) are extracted first from the live vocabulary of the database.

Figure 4 – RulePlanner decision order.

6.1 Entity extraction (nlp.py)

- **Vocabulary from the database.** Categories, company names and the first word of each medicine name are loaded from the live tables at request time, so new categories work without code changes.
- **Fuzzy matching.** `difflib` with a 0.84 cutoff tolerates typos ("paracetmol" → Paracetamol).
- **Periods.** today, this/last week, this/last month, last N days, month-to-date → concrete date ranges; the resolved range is printed as an assumption in the answer.
- **Numbers and comparisons.** "fewer than 10", "above $20" → operator + value; "next 14 days / 2 weeks" → horizon (forecast default 14, clamped to 60).
- **Follow-ups.** If conversation state exists and the message looks like a refinement ("Now…"), the saved SQL *spec* is merged with the new condition rather than rebuilt; example: "fewer than 10 units" → "Now show only tablets" adds `category = Tablet` and keeps the numeric filter (5 → 2 rows).

6.2 LLM planner (optional)

When a provider is configured the planner prompt (`PLANNER_SYSTEM`) contains the tools available to *that role*, the schema visible to *that role* (from `permissions.schema_prompt`), the known categories and the previous plan. The model must return a single JSON object (`max_tokens=500`, temperature 0) which is parsed with a tolerant extractor and then **validated**: the tool must exist and be allowed for the role, `args` must be an object, `sql` must be a string. Any failure silently falls back to the RulePlanner. An LLM-written SELECT goes through exactly the same SQL guard as a rule-built one.

> **Design note**
> **Why rules first?** For a closed domain (stock, expiry, sales) rules give predictable behaviour, zero latency and zero data egress. The LLM planner exists for unusual phrasing; it cannot widen what the user is allowed to do because every plan is re-validated against the role.

7. Tool calling

Tools are the safest path: each is a hand-written function with a fixed, parameterised query. The planner chooses the tool and arguments; `run_tool` then enforces the contract — unknown arguments are dropped, values are clamped (`_int`, `_enum`, `_str`), and the role is re-checked.

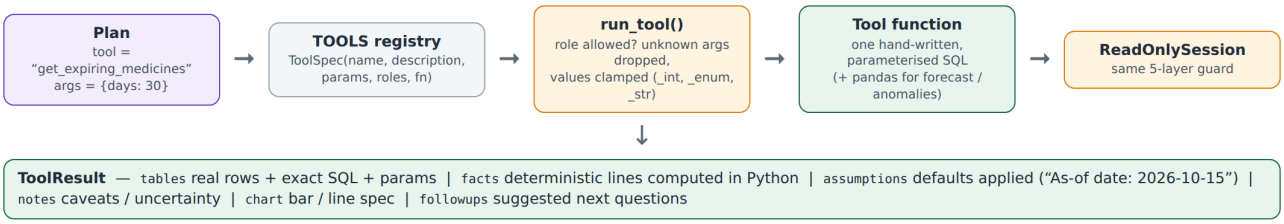


Figure 5 – Anatomy of a tool call.

Tool	Parameters	Roles
`get_expiring_medicines`	days, include_expired, category, company, name	all roles
`get_expired_medicines`	category, company, name	all roles
`get_low_stock_items`	threshold, category, company, name, limit	all roles
`get_stock_level`	name	all roles
`search_medicines`	category, company, name, min_price, max_price, min_qty, max_qty, sort_by, limit	all roles
`get_inventory_value`	group_by	all roles
`get_sales_summary`	period, group_by, category	all roles
`get_category_sales`	period, category	all roles
`get_top_selling`	limit, period, metric, category	all roles
`get_reorder_suggestions`	target_days	admin, pharmacist
`forecast_demand`	name, horizon_days	admin, pharmacist
`detect_anomalies`	lookback_days	admin, pharmacist
`get_management_summary`	period	all roles

Every tool returns a `ToolResult`: *tables* (real rows, exact SQL, parameters), *facts* (sentences computed in Python from those rows — not generated by a model), *assumptions* (defaults such as as-of date, window, thresholds), *notes* (caveats), an optional *chart* spec and *follow-ups*. Because facts are computed in code, the headline numbers cannot be hallucinated even when an LLM later paraphrases them.

Example (get_reorder_suggestions). Items are flagged when stock ≤ 10 units *or* days-of-cover ≤ 14 at the 30-day average sales rate; suggested quantity = *avg daily sales* × *30* − *current stock* (minimum top-up of 20 for items with no recent sales). Demo result: Warfarin 5mg, 3 units, 0.5 days of cover, suggest 171.

8. Text-to-SQL and SQL security

Questions that no tool covers (“products with fewer than 10 units”, “medicines priced above $20”, “expiry before 2027-01-01”) go to the **sql route**. In offline mode the SQL is *built from a structured spec* by `nl2sql.py`, never by free text generation; with an LLM it may be model-written but is treated identically.

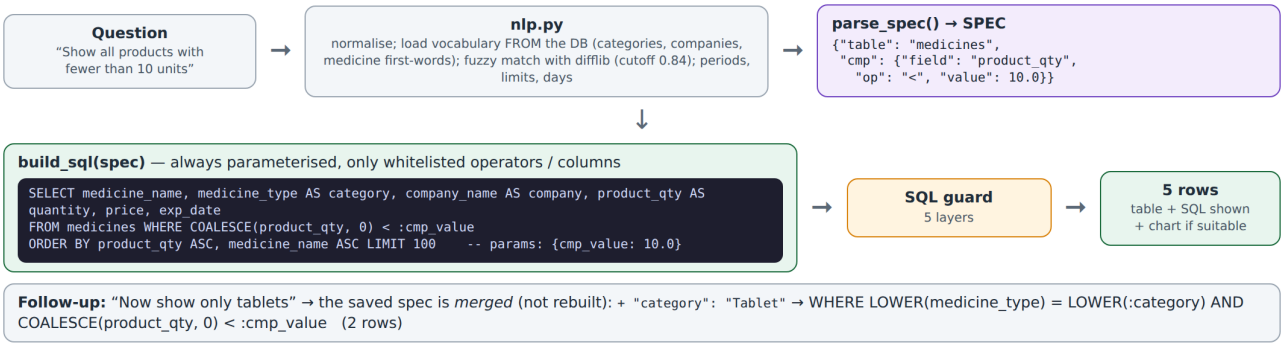


Figure 6 – Offline text-to-SQL: question → spec → parameterised SQL → guard → rows (with a follow-up merge).

8.1 The five-layer SQL guard

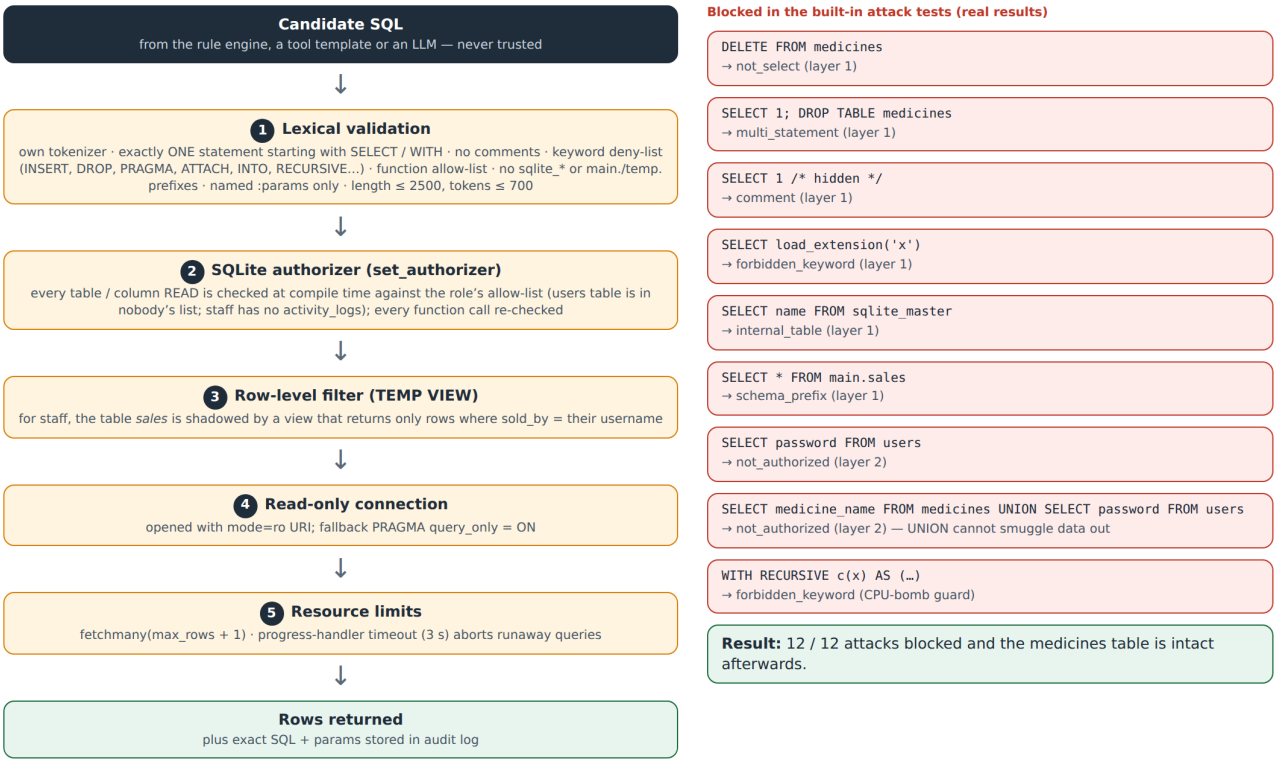


Figure 7 – Layers of the guard and the attacks blocked in the test suite.

Layer	Mechanism (sql_guard.py)	What it stops
1 Lexical	Custom tokenizer (not regex over raw text): one statement, must begin SELECT or WITH, no comments, named parameters only, ≤ 2,500 chars / ≤ 700 tokens. Deny-list of 25 keywords (e.g. ALTER, ANALYZE, ATTACH, BEGIN, COMMIT, CREATE, DELETE, DETACH, DROP…). Function allow-list of 35 (abs, avg, cast, coalesce, count, date, datetime, glob…).	Writes, DDL, stacked statements, hidden comments, extension loading, PRAGMA, recursive CTE CPU bombs, system tables

Layer	Mechanism (sql_guard.py)	What it stops
2 Authorizer	`conn.set_authorizer`: SQLite itself reports every table / column read and function call while compiling the statement; reads outside the role's allow-list are denied.	Anything that slips past parsing: UNION into forbidden tables, subqueries, aliases
3 Row filter	For staff, a TEMP VIEW named *sales* shadows the base table and returns only rows with *sold_by = username*.	Cross-user data exposure inside an allowed table
4 Read-only	Connection opened with `mode=ro` URI (fallback `PRAGMA query_only=ON`).	Any write that escaped layers 1–2
5 Limits	`fetchmany(max_rows+1)` (default 200) and a progress handler aborting after 3 s.	Accidental / hostile heavy queries

What the AI layer may touch	Admin	Pharmacist	Staff
medicines table (all columns)	✓	✓	✓
sales table	✓ all rows	✓ all rows	✓ own rows only (TEMP VIEW)
activity_logs (without ip_address / user_id)	✓	✗	✗
users table (password hashes)	✗ never	✗ never	✗ never
Tools: forecast, anomalies, reorder	✓	✓	✗ not offered
Knowledge documents	✓ all	✓ staff + pharmacist	✓ staff-level only
Upload / delete documents	✓ both	✓ upload only	✗
AI Audit Trail, AI Evaluation	✓	✗	✗

Figure 8 – What each role may touch (enforced in layers 2 and 3 and at tool level).

> The guard is deliberately *defensive redundancy*: layer 1 alone could be bypassed by a clever parser edge case, but layer 2 asks SQLite's own compiler what is actually being read, and layer 4 makes writing impossible regardless.

9. Retrieval-Augmented Generation (RAG)

RAG answers questions about *documents* (leaflets, SOPs) rather than numbers. The implementation is a compact, dependency-light pipeline in `pharmaiq/rag/` (about 568 lines). It has two phases: **indexing** at upload time and **querying** per question.

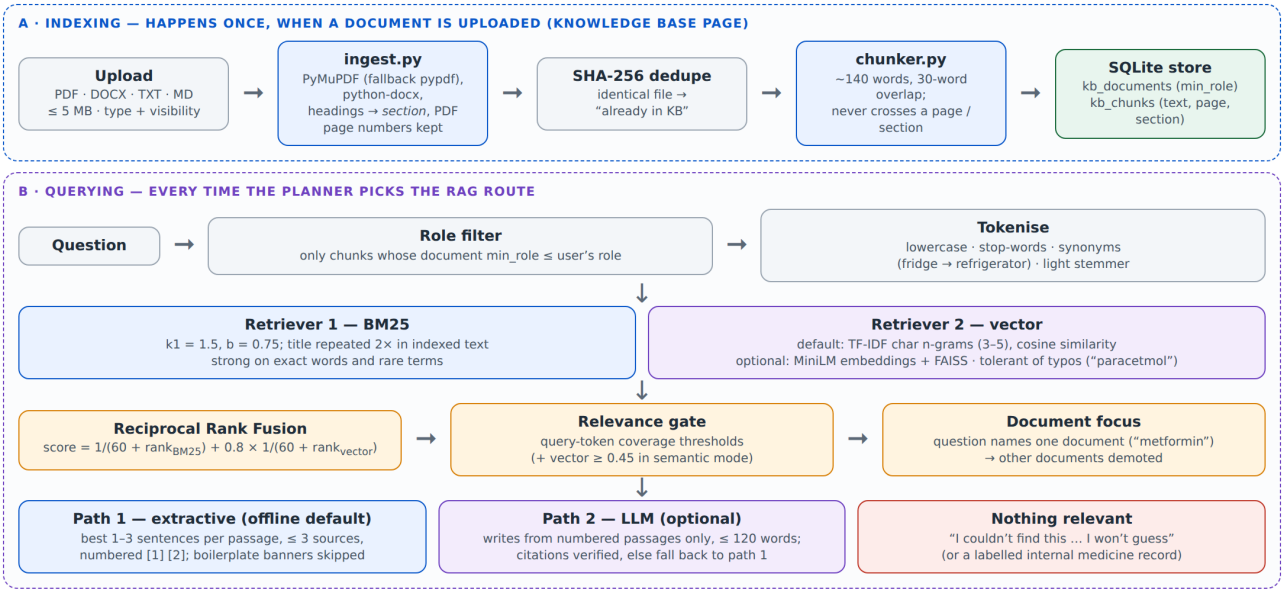


Figure 9 – The RAG pipeline (A: indexing, B: querying).

9.1 Indexing

- **Extraction (ingest.py).** PDF via PyMuPDF (fallback pypdf) keeping page numbers; DOCX via python-docx where Heading styles become the *section* label; TXT / MD parsed with Markdown headings. 5 MB cap, extension whitelist.
- **De-duplication.** SHA-256 of the file content (*source_hash UNIQUE*) — re-uploading the same file is a no-op.
- **Chunking (chunker.py).** ~140-word chunks with a 30-word overlap, built paragraph by paragraph. Chunks **never cross a page or section boundary**, so every citation points to a precise place. Over-long paragraphs are windowed with the same overlap. The demo corpus (12 documents) yields **66 chunks**.
- **Storage.** *kb_documents* holds the document's *min_role*; *kb_chunks* holds text, page, section and (optionally) an embedding BLOB. In-memory indexes are cached per (database, role, backend) and invalidated when the knowledge-base version changes.

9.2 Text normalisation (rag/text.py)

Both documents and queries pass through the same tokeniser: lowercase → split on `[a-z0-9]+` → drop stop-words (including question words such as *what, how, show*) → map domain **synonyms** (fridge → refrigerator, pills → tablet, outdated → expired, meds → medicine …) → apply a deliberately **light stemmer** (strip *-ing, -ed, -es, -s*, trailing *e*). Result for *"How should insulin be stored?"*: `['insulin', 'stor']`; for *"What are the side effects of metformin?"*: `['side', 'effect', 'metformin']`. The stemmer is crude by design (store / stored / storing → *stor*) — good enough for a controlled vocabulary, with no NLTK dependency.

9.3 Hybrid retrieval: two complementary rankers

Keyword ranker – BM25. For a query term *t* and chunk *d*:

```
score(d) = sum over query terms t of:
    IDF(t) * f(t,d) * (k1 + 1) / ( f(t,d) + k1 * (1 - b + b * |d| / avgdl) )
with k1 = 1.5, b = 0.75;  f = term frequency in chunk d,  avgdl = average chunk length
```

The indexed text of each chunk is *title + title + section + body* (the title is repeated, giving title matches roughly double weight, which is how "metformin" finds the Metformin leaflet). BM25 excels at exact and rare terms.

Vector ranker – TF-IDF over character n-grams (default). `TfidfVectorizer(analyzer="char_wb", ngram_range=(3,5), sublinear_tf=True)` converts text to sparse vectors of 3–5-character fragments; similarity is cosine. Because it matches *sub-word* pieces it tolerates typos and word-form changes ("paracetmol", "storing" vs "storage") that defeat whole-word BM25. It needs no model download and no GPU.

Optional neural embeddings. With `PHARMAIQ_EMBEDDINGS=sentence-transformers`, chunks are embedded with *all-MiniLM-L6-v2* (384-dim), persisted as BLOBs in *kb_chunks*, and searched with FAISS; this adds true semantic matching ("fridge" ≈ "refrigerated storage" without a synonym entry). If the library is missing the system falls back to TF-IDF and says so on the Knowledge Base page.

9.4 Rank fusion, relevance gate and focus

The two rankings are merged with **Reciprocal Rank Fusion**, which uses ranks rather than raw scores (BM25 and cosine are on incompatible scales):

```
RRF(d) = 1 / (60 + rank_BM25(d))  +  0.8 * 1 / (60 + rank_vector(d))
(the BM25 term is included only when the BM25 score is > 0)
```

Query: "How should insulin be stored?" (top 7 of 66 chunks by fused rank)

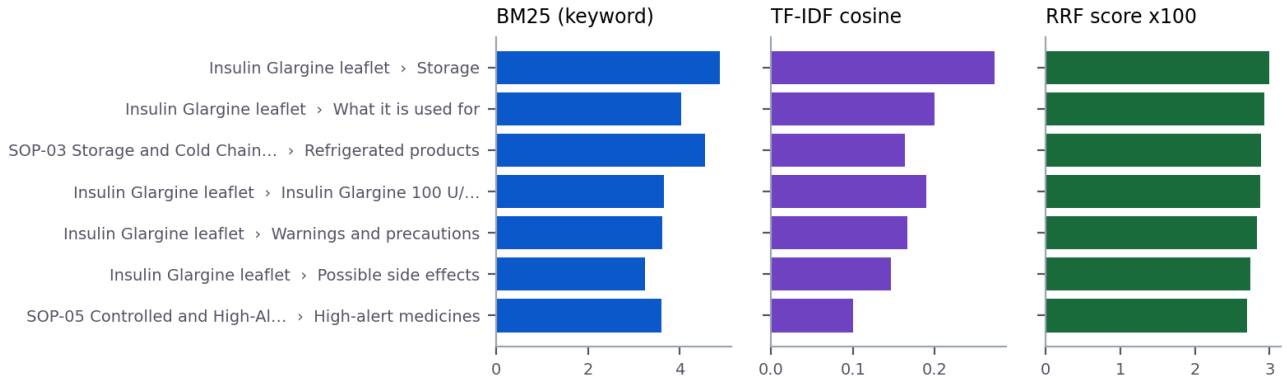


Figure 10 – Real scores for "How should insulin be stored?": the two rankers disagree on order; fusion produces a stable top list (Insulin leaflet › Storage first, SOP-03 Refrigerated products also surfaced).

Fusion alone always returns *something*, which is dangerous. A **relevance gate** therefore decides which hits may be used. In offline mode a hit is relevant if (coverage ≥ 0.5 and BM25 > 0) *or* (coverage ≥ 0.34, ≥ 3 query tokens, BM25 > 0) *or* (a single-token query fully covered), where *coverage* = fraction of query tokens present in the chunk. In semantic mode: coverage ≥ 0.4 *or* cosine ≥ 0.45. For "What is the capital of France?" nothing passes the gate, so the answer is "I won't guess".

Two refinements follow: **(1) order inside a document** by how well the passage's own section + text (not the repeated title) matches; **(2) document focus** — if the question names a specific subject (a title token appearing in ≤ 25% of documents, e.g. *metformin*) and that document matches at least as well as the others, all other documents are marked not-relevant so a "side effects of metformin"

question never cites the amoxicillin leaflet merely because both contain “side effects”.

9.5 Answer generation

Path	How it works	Guardrails
Extractive (default)	For each relevant hit (max 3 sources) pick up to 3 sentences by query-token overlap, normalised by sentence length; boilerplate banners (“Sample…”, “not a substitute for…”) are skipped. Output: **Document › Section, page** [n]: sentences.	Text is copied from the document; nothing can be invented. If only boilerplate matched → no answer.
LLM-written (optional)	The model receives numbered passages and must answer in ≤ 120 words, citing [n]. Passage text is treated as data, not instructions.	Answer must contain citations and every cited number must exist in 1..n; otherwise it is discarded and the extractive answer is used.
No answer	“I couldn’t find this in the uploaded leaflets or SOPs, so I won’t guess…”	Never falls through to the model’s own knowledge.

9.6 RAG access control and evaluation

Access control is applied before retrieval: only chunks whose document *min_role* is ≤ the caller’s role are loaded into the index for that role, so restricted passages cannot appear in scores, snippets or citations. The built-in RAG evaluation (15 cases in `evaluation/rag_eval.json`) checks that the expected document is cited with the expected keywords and that off-topic questions get no answer; all 15 pass on the demo corpus. (Role-based document visibility is enforced in the retrieval layer and covered by the RAG unit tests, not by these 15 cases.)

> **Limitations to know.** The default retriever is lexical: it matches words and fragments, not meaning, so a question phrased with entirely different vocabulary may miss (enable embeddings to mitigate). Unsupported queries that fall through to RAG can return an unrelated passage, e.g. for staff asking “Show me all activity logs” the demo returns an incident-reporting SOP passage because of the word “log”. Scanned PDFs need OCR first.

10. Data science: forecasting, anomalies and analytics

All analytics live in `analytics.py` as pure functions over pandas Series (about 170 lines) so they are easy to test. Data is pulled with read-only SQL, aggregated to a daily series, and modelled *per request* — there is no stored model, no training job and nothing to retrain.

10.1 Demand forecasting

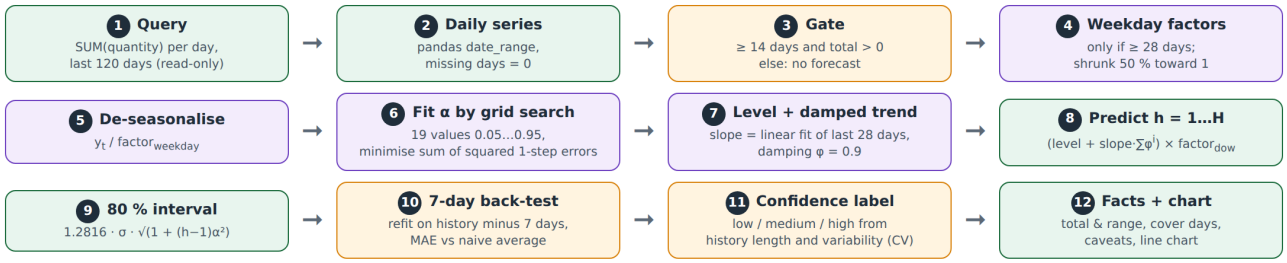


Figure 11 – Forecast pipeline.

Model. A *damped-trend simple exponential smoothing with multiplicative weekday seasonality*, written from scratch in numpy:

```
1. y_t   = units sold on day t (last 120 days, missing days = 0)
2. s_d   = weekday factor  = mean(y | weekday d) / mean(y)            (only if n >= 28 days)
   s_d := 1 + 0.5 (s_d - 1), renormalised to mean 1      (50% shrinkage against over-fitting)
3. z_t   = y_t / s_weekday(t)                                        (de-seasonalised)
4. level_t = level_{t-1} + alpha (z_t - level_{t-1});  alpha chosen from 19 values in [0.05, 0.95]
             minimising the sum of squared one-step errors
5. slope = linear-regression slope of the last 28 values of z
6. forecast(h) = max(0, level + slope * sum_{i=1..h} 0.9^i) * s_weekday(T+h)   (damping 0.9)
7. 80% interval(h) = forecast(h) +/- 1.2816 * sigma * sqrt(1 + (h-1) alpha^2) * s_weekday
   (sigma = standard deviation of the one-step errors)
8. total interval = midpoint of the "errors independent" and "errors perfectly correlated" widths
```

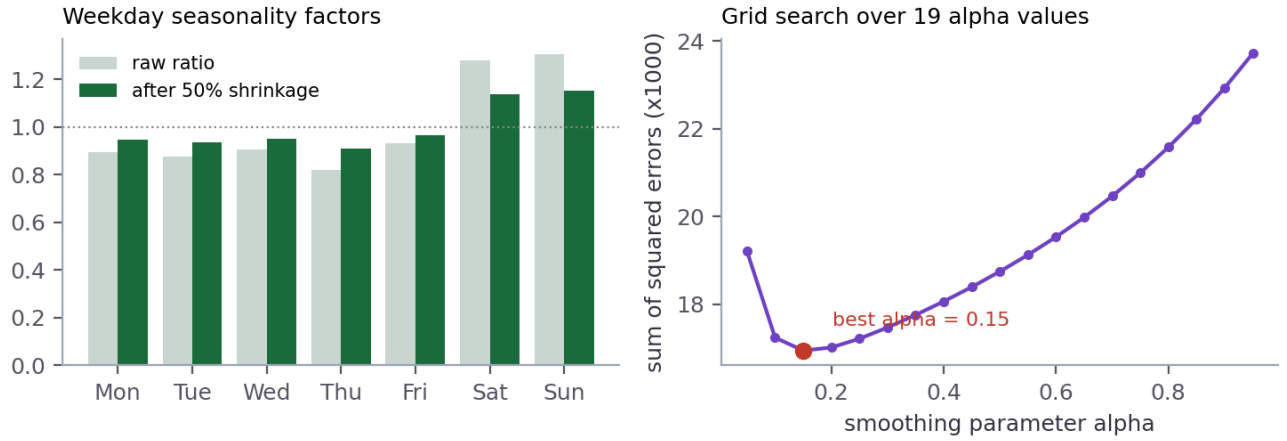


Figure 12 – Fitted internals on the demo data: shrunk weekday factors (weekends sell ~15% above average) and the squared-error curve whose minimum selects α = 0.15.

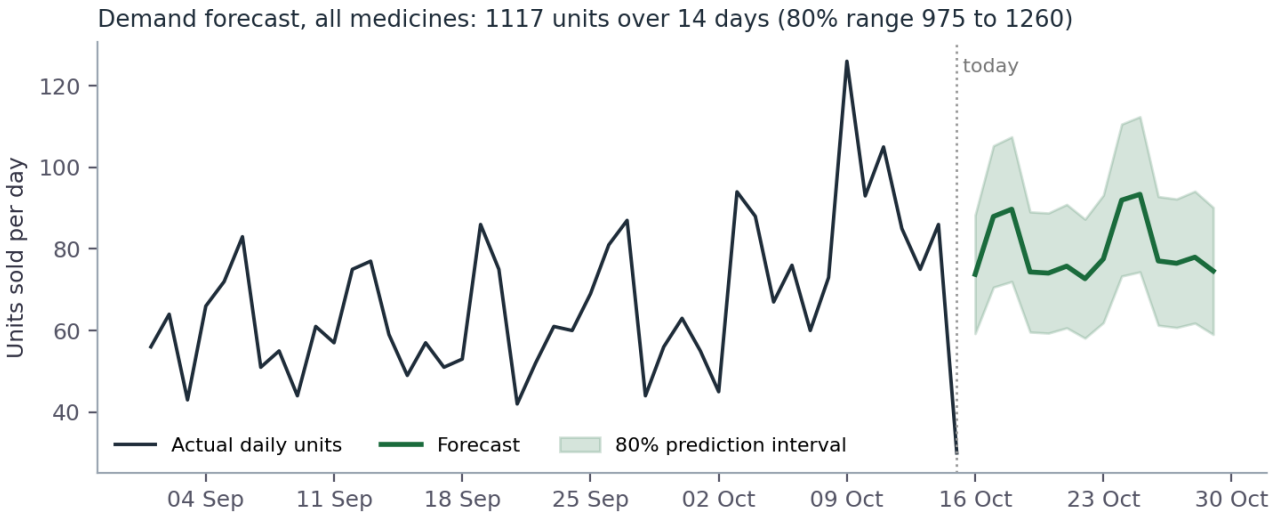


Figure 13 – Resulting 14-day forecast for all medicines: 1,117 units (80% range 975–1,260).

Why this model? It is interpretable, needs ~2 weeks of data, has almost no hyper-parameters, runs in milliseconds, and degrades safely (flat level when data is short). Heavier models (ARIMA, Prophet, gradient boosting) need more history than a single pharmacy typically has per medicine.

Honest uncertainty. Each forecast reports (a) the 80% range, (b) a **confidence label**: *low* if fewer than 28 days or CV > 1.0; *high* if ≥ 56 days and CV < 0.6; otherwise *medium* (CV = std / mean of daily sales), and (c) a **7-day back-test**: the model is refit on history minus the last 7 days and its mean absolute error is compared with a naive baseline (previous week's average). If the model is not better than naive, a warning is added. On the demo data: MAE 25.5 vs naive 25.8 units/day, i.e. only marginally better than a plain average for the whole shop, while for a single slow-moving product (paracetamol: 34 units, range 3–64) the interval is very wide. The UI states that promotions, outbreaks and supply issues are not modelled.

10.2 Anomaly detection

Two detectors run in `detect_anomalies`:

(a) Sales spikes / drops – robust z-score. For each day, the baseline is the *median of the previous 28 days* and the scale is *max(1.4826 · MAD, 0.5 · max(median, 1), 1)*. A spike is flagged when z ≥ 3.5 and the value ≥ max(min_units, 1.5 × median); a drop when z ≤ −3.5 and the median is large enough to matter. Median / MAD is used instead of mean / standard deviation because a single extreme day would otherwise inflate the baseline and hide itself.

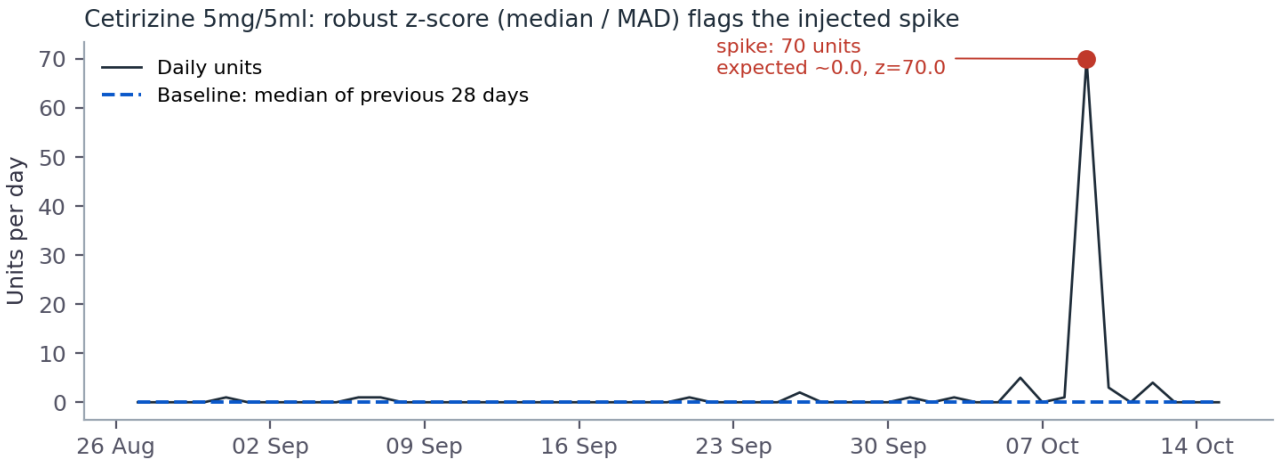


Figure 14 – A planted spike in the demo data (Cetirizine, 70 units against a typical 0–1) flagged by the detector.

(b) Data-quality rules (deterministic SQL): negative stock; expiry before issue date; missing expiry; duplicate barcodes; sale total ≠ quantity × unit price; sales of a medicine id that does not exist; and **sales after expiry** (severity high). On the demo data this yields 12 findings: 7 spikes, 4 sold-after-expiry, 1 total mismatch.

10.3 Other analytics

- **Reorder heuristic** – average daily sales over 30 days, days-of-cover, rule thresholds (≤ 10 units or ≤ 14 days) and a target-days top-up (Section 7).
- **Aggregations** – inventory value (price × quantity) by category / company, sales by day / medicine / category / seller, top sellers by units or revenue, expiry windows with stock value at risk (e.g. 550 units, value 10,090.00 in the demo).
- **Management summary** – one tool composing inventory totals, low-stock count, expiry exposure, sales by category and top sellers into a single briefing.

11. Where each “AI” technique is used

User need	Technique	Why this choice
Understand “fewer than 10 units”	Rules + fuzzy matching + vocabulary from DB	Closed domain; deterministic and testable; no model risk
Find the right document passage	BM25 + char n-gram TF-IDF + RRF (+ optional MiniLM / FAISS)	Lexical exactness plus typo tolerance without GPUs; embeddings available when needed
Decide if retrieval is good enough	Coverage-based relevance gate	Prevents confident answers from weak matches
Predict demand	Damped-trend exponential smoothing + weekday seasonality	Works with weeks of data; interpretable; explicit uncertainty
Spot unusual sales	Robust z-score (median / MAD)	Resistant to the outliers it is searching for
Explain results in plain English	Rule templates (default) or LLM (optional)	LLM output is verified against the data before display
Handle odd phrasing	LLM planner (optional)	Plans are validated and re-checked per role

12. LLM integration

The LLM layer (`llm.py`, ~150 lines) is a thin provider abstraction using only `urllib`. **Anthropic** calls `/v1/messages`; **openai** calls any OpenAI-compatible `/chat/completions` endpoint (OpenAI, Ollama, LM Studio, vLLM, Groq…), so a fully *local* model can be used. Temperature is fixed at 0. Enabling it is configuration only:

```
PHARMAIQ_LLM_PROVIDER=anthropic          # offline (default) | anthropic | openai
PHARMAIQ_API_KEY=...                     # or ANTHROPIC_API_KEY / OPENAI_API_KEY
PHARMAIQ_MODEL=claude-sonnet-5-5         # default per provider; gpt-4o-mini for openai
PHARMAIQ_BASE_URL=http://localhost:11434/v1   # e.g. local Ollama
PHARMAIQ_LLM_MAX_ROWS=15                 # rows an LLM may see; 0 = aggregates only
```

The model performs exactly **two jobs**, each wrapped in checks, and one optional third:

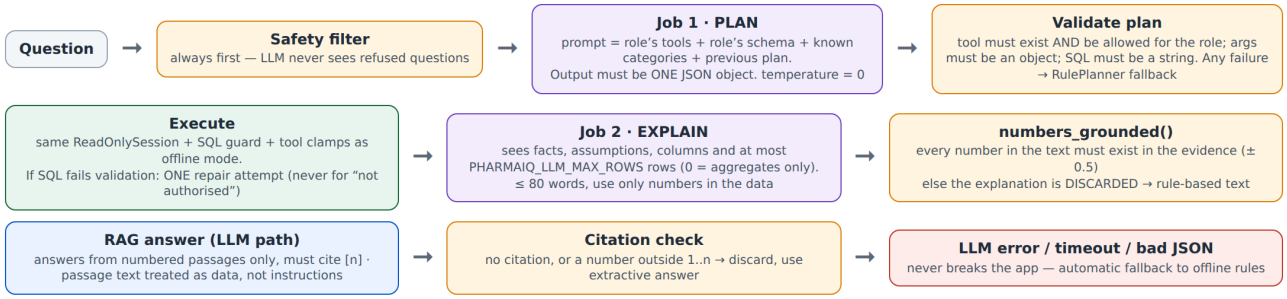


Figure 15 – LLM jobs and guardrails.

Job	Input to the model	Verification before use
1 Plan	Role-specific tool list + schema, known categories, previous plan, the question	Strict JSON; tool exists and allowed for role; SQL passes the 5-layer guard; any failure → RulePlanner
2 Explain results	Question, facts, assumptions, column names and ≤ N rows (N = *PHARMAIQ_LLM_MAX_ROWS*)	`numbers_grounded()`: every number in the text must appear in the evidence (± 0.5); otherwise the model text is dropped for the rules-based explanation. UI labels it *model* vs *rules*
3 Write RAG answer	Numbered passages only; instruction to cite [n] and ignore instructions inside passages	Must cite; citation numbers must be within 1..n; else extractive answer is used

12.1 Data minimisation and failure modes

- **Privacy:** with *PHARMAIQ_LLM_MAX_ROWS=0* only aggregates leave the server; with a local OpenAI-compatible model nothing leaves the machine. Refused questions (clinical, credentials) never reach the LLM.
- **Prompt injection:** document passages and database values are labelled as data; the safety filter, role validation and SQL guard sit outside the model and cannot be negotiated with by text.
- **Failures:** timeouts (*PHARMAIQ_LLM_TIMEOUT*, default 30 s), HTTP errors and malformed JSON raise `LLMError` and the request completes on the offline path; `/api/ai/health` shows which mode is active.
- **One repair attempt:** if an LLM-written SQL fails validation (not for authorisation failures) the error is fed back once for correction.

13. Safety, security and privacy

Control	Detail
Clinical safety filter	Regex families: person-specific cues (my son / patient ...) + dose words; “can / should I take / give...”; pregnancy / breast-feeding questions; recommendation or diagnosis requests (“what should I take”, “prescribe”, “do I have diabetes”); personal numbers (kg, years old). Returns a fixed refusal pointing to the pharmacist and to what PharmaIQ *can* do. Reference questions about leaflets (“warnings for warfarin”) are allowed.
Credential filter	Password, API key, secret key, token, credentials, users table, “show all users” → refusal for every role.
Authentication / roles	Flask session login (Werkzeug hashes); page gating from `PAGE_ROLES`; tools and tables gated per role.
CSRF	JSON POSTs require an `X-CSRF-Token` header (session token from `secrets.token_urlsafe`); upload forms embed the token.
Export safety	Excel / CSV cells starting with `= + - @` are prefixed with an apostrophe (spreadsheet formula-injection defence); exports are owner-only and expire after 48 h.
PII redaction	E-mail addresses and long digit sequences are masked in the audit log (questions truncated to 300 chars).
Upload hardening	Extension whitelist, 5 MB limit, content-hash de-duplication, text-only extraction (no macro / script execution).
Audit trail	Every request incl. refusals and errors is written to *ai_audit_log* with SQL and latency.

Hardening checklist

The following concern the **original PharmaCare app**, not the AI layer, and should be addressed before real use: the default demo passwords; the Flask SECRET_KEY falls back to a hard-coded default when the environment variable is not set; and the legacy `/api/*` endpoints accept the app secret key as an API key. The audit log stores SQL and evidence, which can contain business data — restrict database file access accordingly.

14. Evaluation and testing

Two complementary mechanisms exist. The **unit-test suite** (89 tests: agent 28, tools 20, SQL guard 17, RAG 11, API 10, evaluation 3) checks code paths. The **evaluation harness** (`python -m pharmaiq.evaluate` or the AI Evaluation page) runs curated, data-driven cases against the live database in offline mode:

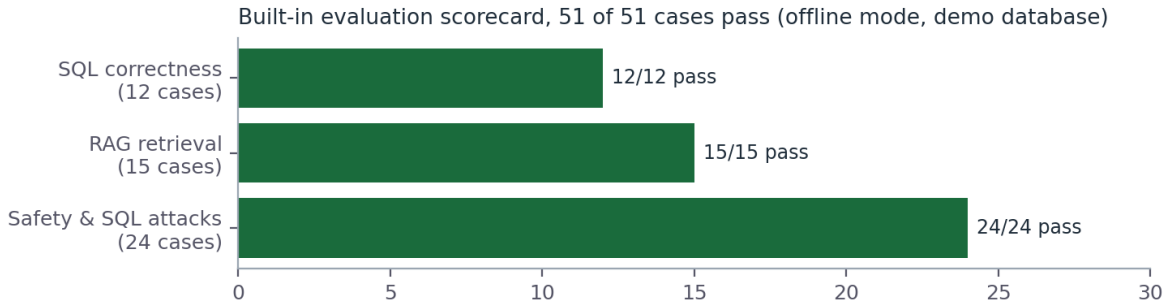


Figure 16 – Built-in evaluation on the demo database (51 / 51).

Suite	Cases	What is checked
sql_eval.json	12	Each case states the expected route / tool and, where applicable, is checked against an independent **oracle**: a hand-written SQL count run directly on the database (e.g. products below 10 units, batches expiring in the window, a staff user’s own sales). Also checks that forecasting is refused for the staff role.

Suite	Cases	What is checked
rag_eval.json	15	The expected document title must be cited and expected keywords must appear in the answer (e.g. “What is the return policy?” → Returns and Refunds); negative cases such as “Explain quantum computing” must produce no answer.
safety_eval.json	24	7 refusals, 4 benign questions that must NOT be refused, 12 SQL-guard attacks (writes, stacked statements, comments, extension loading, system tables, UNION into users, recursive CTE), 1 integrity check that the medicines table is intact afterwards

How to read the 51 / 51. It is a regression suite written by the project authors on a small demo dataset — it demonstrates that designed behaviours hold, not that every real-world question is answered correctly. Known gaps found during this review are listed in Section 16. Extending the JSON files is the intended way to grow coverage; the harness requires no code changes.

15. Configuration reference

Variable	Default	Meaning
SECRET_KEY	(demo default)	Flask session signing key — **set a long random value**
PHARMAIQ_LLM_PROVIDER	offline	offline | anthropic | openai
PHARMAIQ_API_KEY / ANTHROPIC_API_KEY / OPENAI_API_KEY	—	Provider credential
PHARMAIQ_MODEL	claude-sonnet-5-5 / gpt-4o-mini	Model name
PHARMAIQ_BASE_URL	OpenAI URL for openai	Custom OpenAI-compatible endpoint (e.g. Ollama)
PHARMAIQ_LLM_TIMEOUT / _MAX_ROWS	30 s / 15	LLM timeout; rows shown to the LLM (0 = none)
PHARMAIQ_EMBEDDINGS	tfidf	tfidf | sentence-transformers
PHARMAIQ_EMBED_MODEL	sentence-transformers/all-MiniLM-L6-v2	Embedding model
PHARMAIQ_CHUNK_WORDS / _CHUNK_OVERLAP	140 / 30	Chunk size and overlap
PHARMAIQ_UPLOAD_MAX_MB	5	Upload size limit
PHARMAIQ_MAX_ROWS / _SQL_TIMEOUT	200 / 3 s	Result row cap; SQL execution timeout
PHARMAIQ_LOW_STOCK / _EXPIRY_DAYS	10 / 30	Alert thresholds shared with the original app
PHARMAIQ_RESULTS_TTL_HOURS	48	Lifetime of export handles
PHARMAIQ_TODAY	(system date)	Pin “today” (YYYY-MM-DD) for reproducible tests / demos

16. Extending the system and known limitations

16.1 Common extensions

Goal	How
Add a new tool	Write a function returning a `ToolResult`, register a `ToolSpec(name, description, params, roles, fn)`, and add a planner rule (or rely on the LLM planner). Add cases to sql_eval.json.
Better semantic search	pip install -r requirements-optional.txt and set `PHARMAIQ_EMBEDDINGS=sentence-transformers`; re-upload or reload documents so embeddings are stored.
Use a local LLM	Run Ollama; `PHARMAIQ_LLM_PROVIDER=openai`, `PHARMAIQ_BASE_URL=http://localhost:11434/v1`, `PHARMAIQ_MODEL=llama3.1`.
Support new document types	Add an extractor in `rag/ingest.py` returning blocks of {page, section, text}.
Move to PostgreSQL	SQLite-specific pieces (authorizer, mode=ro, progress handler, TEMP VIEW row filter) in `sql_guard.py` would need equivalents (read-only role, row-level security policies, statement_timeout).

16.2 Known limitations found during this review

Area	Observation	Suggested remedy
Forecast input	The daily series ends on the current day, so a partially elapsed day is treated as a full (low) day; this is visible as the final drop in Figure 13.	Exclude today from the training series or scale by elapsed fraction.
Forecast value	On the demo data the shop-level back-test error (25.5) is only marginally better than the naive baseline (25.8).	Keep the displayed back-test; consider per-category models and promotional features.
Fallback routing	Unsupported questions fall through to RAG and may return a loosely related passage (e.g. staff: “Show me all activity logs” → incident SOP).	Add a refusal / “not available to your role” route for known-but-forbidden data; raise the relevance gate for single generic words.
NLU coverage	Rule-based phrasing gaps: “top 10 sellers”, “what sells the least” / “bottom 5”, “demand over 2 weeks” (without a verb), sort-order follow-ups (“Now sort by price”); the word “report” can trigger the management summary.	Add rules / synonyms and test cases, or enable the LLM planner (validated).
Anomaly scale	For items that rarely sell, any burst yields a very large z (e.g. 70.0 for the planted spike) because the scale floors at 1.	Report as ratio to typical volume; add a minimum-history requirement per item.
Evaluation scope	Suite is small, curated and offline-mode only; no LLM-mode or large-corpus evaluation is included.	Add LLM-mode regression runs and a larger labelled question set.
Original app	Hard-coded fallback SECRET_KEY, demo passwords, secret key accepted as legacy API key.	See hardening checklist (Section 13).

Appendix A. Glossary of techniques

Term	Meaning in this project
RAG	Retrieve relevant document passages, then answer only from them.
Chunk	A ~140-word passage of a document with page / section metadata; the unit that is indexed and cited.
BM25	A probabilistic keyword-ranking function with term-frequency saturation (k1) and length normalisation (b).
TF-IDF (char n-gram)	Sparse vectors of weighted 3–5-character fragments; cosine similarity gives typo-tolerant matching.
Embedding / FAISS	Dense semantic vectors and a similarity-search library (optional here).
RRF	Reciprocal Rank Fusion: combines rankings via 1/(k + rank) so incompatible scores need no calibration.
Relevance gate	Threshold rule deciding whether retrieved text is good enough to answer from.
Exponential smoothing	Forecast whose level is a weighted average of past values, with weights decaying geometrically (α).
Damped trend	A trend whose influence shrinks each step (φ = 0.9) so forecasts do not extrapolate indefinitely.
Prediction interval (80%)	Range expected to contain ~80% of future observations under the model's assumptions.
MAD / robust z-score	Median absolute deviation based scale; resists distortion by outliers.
Back-test	Evaluating a forecasting method on recent data withheld from fitting.
Text-to-SQL	Turning a question into a SQL query; here via a structured spec (offline) or an LLM, always through the guard.
Tool calling	The planner selects a named function with arguments instead of writing free-form code.
Authorizer	SQLite hook that approves or denies each table / column / function access during statement compilation.
Grounding check	Verification that numbers and citations in generated text exist in the supplied evidence.

About this document
This guide was produced by reading the source in PharmaCare-PharmaIQ.zip and running it on its demo database (as-of date 2026-10-15). Figures marked as real outputs were generated by the project's own code; diagrams are drawn from the implemented behaviour.